Zero-Overhead Compile-Time Pipeline Synthesis via Nested Inversion Typestacks

1. Introduction & Motivation

In deeply embedded systems engineering, hardware constraints dictate strict boundaries for software design. Early attempts to unify hardware access layers, such as abstraction frameworks for microcontroller communication mediums (e.g., VirtualPins, Dec 2017), demonstrated that traditional OOP models fail to achieve true cross-platform and cross-framework generalization without performance degradation.

The central problem lies in the structural static anatomy of the code itself: a human-written, monolithic execution block consistently outperforms abstract, modular code structures built using standard runtime designs. To bridge this gap, HAPI shifts the entire burden of composition, configuration, and verification from runtime execution to the compilation phase, generating deterministic, zero-overhead execution pipelines.

2. The Problem Landscape

1. **Platform Coupling and Micro-Management:** Existing drivers inherently couple hardware management to concrete runtime platforms. Peripheral logic is systematically rewritten for every unique physical interface variant. Traditional dynamic dispatch (`v-tables`) is explicitly prohibitive due to pointer latency and dynamic memory requirements.
2. **Feature Creep and Resource Bleeding:** Expanding functionality typically introduces conditional runtime flags and state checking. Even when a feature is unused, its supporting scaffolding remains resident in the compiled binary, creating a strict developmental ceiling.
3. **Structural Optimization Trade-offs:** Achieving deep modularity via standard inheritance results in structural code duplication or deep structural nesting that prevents the compiler from performing aggressive inlining and register allocations.

3. The Architecture: Compile-Time Geometry

To resolve the syntactic complexity of traditional template formulations—such as deeply nested `template<template<typename> class>` constructs which fail to isolate configuration parameters from derivation paths—HAPI introduces a strict single-parameter inversion mechanism via a nested `Part` template structure:

```
template <typename Parent>
struct Component {
  template <typename O>
  struct Part : public O {
    // Linear composition logic resides here
  };
};
```

This acts as the compile-time equivalent of **partial binding**. All architectural configuration parameters are pre-applied to the component in an isolated phase.

Aggressive Functional Elimination & Fallback APIs

If a feature is omitted from the declarative composition stack, it ceases to exist across three domains:

- **Zero Internal Code Footprint:** The unused component logic is absent from the derivation tree.
- **Zero Calling Overhead:** Invocations to omitted features are intercepted by a static `constexpr` Fallback API containing an empty body or returning a predefined literal value, completely erasing the call-site instructions.
- **Zero Data Footprint:** Storage invariants are tightly encapsulated within each `Part` layer; eliminating the component instantly drops its associated data allocations from the memory map.

4. The Chain Combinator and Topological Collapse

The `Chain` combinator acts as a type-level folding operator over a flat list of component types, satisfying the algebraic equivalence:

$$\mathrm{Chain}\langle A, B, C\rangle \iff A{::}\mathrm{Part}\langle B{::}\mathrm{Part}\langle C\rangle\rangle$$

Because `Chain` evaluates to a fully compliant HAPI component by exposing its own inner `Part<T>` template, it achieves full algebraic closure, permitting arbitrary nesting and aliases:

$$\mathrm{Chain}\langle \mathrm{Chain}\langle A, B\rangle, C\rangle \iff A{::}\mathrm{Part}\langle B{::}\mathrm{Part}\langle C\rangle\rangle$$

The Folding Implementation

The structural collapse is governed by a recursive template expansion within the `Chain` combinator:

```
template <typename O, typename... OO>
struct Chain {
  template <typename T>
  struct Part : public O::template Part<typename Chain<OO...>::template
Part<T>> {
    using Base = typename O::template Part<typename Chain<OO...>::template
Part<T>>;
  };
};

// recursion base case
template <typename O>
struct Chain<O> {
  template <typename T>
  struct Part : public O::template Part<T> {
    using Base = typename O::template Part<T>;
  };
};
```

Linear Constructor Forwarding and Storage Invariants

To ensure zero-overhead construction and contiguity in the stack, all intermediate components must execute perfect forwarding to their `Base`, providing dual construction paths for components with state:

```
#include <utility>

struct StatefulComponent {
  template <typename O>
  struct Part : public O {
    using Base = O;
    int local_data;

    template <typename... Args>
    Part(int data, Args&&... args)
      : Base(std::forward<Args>(args)...), local_data(data) {}

    template <typename... Args>
    Part(Args&&... args)
      : Base(std::forward<Args>(args)...), local_data(0) {}
  };
};

struct StatelessComponent {
  template <typename O>
  struct Part : public O {
    using Base = O;

    template <typename... Args>
    Part(Args&&... args)
      : Base(std::forward<Args>(args)...) {}
  };
};
```

5. Empirical Verification and Architectural Impact

To validate the theoretical assertions of zero-overhead pipeline synthesis, static inheritance, and aggressive functional erasure via the `Chain` combinator, a firmware implementation targeting the Microchip ATmega328P architecture (8-bit AVR RISC) was analyzed using `avr-objdump`.

The test isolates the `main` execution pipeline across two discrete operational states to observe the compiler's behavior regarding register mapping, compile-time address resolution, and structural consolidation.

5.1 Scenario A: Active Pipeline Infrastructure (`with functions active.txt`)

When the pipeline's operational and telemetry features are integrated as distinct linear layers, the compiler resolves all inter-component dependencies statically at compile time.

```
00000484 <main>:
 484:   78 94           sei
 ...
 57c:   8c e1           ldi r24, 0x1C   ; [cite_start]Load low byte of
Print object (0x011C) [cite: 117]
 57e:   91 e0           ldi r25, 0x01   ; Load high byte of Print object
(0x011C)
 580:   0e 94 91 01     call    0x322   ; [cite_start]Static Resolution:
Print::write(char const*) [cite: 118]
 584:   65 e1           ldi r22, 0x15   ; [cite_start]Load next string
pointer offset [cite: 119]
 ...
 58c:   0e 94 91 01     call    0x322   ; [cite_start]Static Resolution:
Print::write(char const*) [cite: 122]
 ...
 598:   0e 94 91 01     call    0x322   ; [cite_start]Static Resolution:
Print::write(char const*) [cite: 126]
 5a0:   0e 94 71 01     call    0x2e2   ;
[cite_start]HardwareSerial::flush() [cite: 128]
```

Mechanical Observations (Active State)

-

Zero Runtime Indirection: The base address of the synthesized target component (`0x011C`) is coupled directly into the working register pair `r25:r24` using immediate addressing (`ldi`). No virtual pointer (`vptr`) or runtime dispatch table (`vtable`) lookup is executed, proving complete elimination of OOP runtime overhead.

-

Linear Call Geometry: The pipeline emits strict, discrete inline sequences. Arguments for specialized string data references (at RAM boundaries `0x0112`, `0x0115`, and `0x0118`) are loaded directly into the `r23:r22` register pairs prior to executing each direct opcode `call 0x322`.

5.2 Scenario B: Structural Block Consolidation via Typestack Inversion (`functions deactivated.txt`)

When specific tracking or element configurations are stripped, the compiler unifies the resulting static stack geometry to compress operational throughput rather than generating empty instruction loops.

```
00000466 <main>:
 466:   78 94           sei
 ...
 55a:   42 e0           ldi r20, 0x02   ; [cite_start]Setup block size
length parameters [cite: 47]
 [cite_start]55c:   50 e0           ldi r21, 0x00   [cite: 48]
 55e:   62 e1           ldi r22, 0x12   ; [cite_start]Base pointer to
```

```
consolidated array [cite: 49]
 [cite_start]560:   71 e0           ldi r23, 0x01   [cite: 50]
 562:   86 e1           ldi r24, 0x16   ; [cite_start]Object destination
address [cite: 51]
 564:   91 e0           ldi r25, 0x01
 566:   0e 94 84 00     call    0x108   ; [cite_start]Aggregated Call:
Print::write(unsigned char const*, unsigned int) [cite: 52]
 56e:   0e 94 71 01     call    0x2e2   ;
[cite_start]HardwareSerial::flush() [cite: 54]
```

Mechanical Observations (Consolidated State)

-

Topological Collapse to Bulk Write: The sequence of three separate discrete calls (`call 0x322`) is entirely eliminated. Because compile-time resolution guarantees layout contiguity, the compiler collapses the independent stream interactions into a single bulk transfer routine via `call 0x108`.

-

Register Overhead Minimization: Instead of continuously re-indexing and issuing distinct calling context shifts , the configuration passes the size (`0x0002`) and array location parameters directly into the hardware registers, drastically minimizing pipeline setup overhead.

- **Flash Footprint Reduction:** This optimization drastically compresses the execution loop segment. The return vector for the microsecond timing loop (`<micros>`) transitions inward from offset `0x5b8` down to `0x572`, demonstrating significant flash savings without modifying the abstract declarative architecture.

5.3 Architectural Impact Matrix

Implemented Performance Metric	Active Pipeline State	Deactivated Fallback State
Runtime Polymorphism Cost	**0 cycles** (Direct Static Resolution)	**0 cycles** (Direct Static Resolution)
Call Footprint in Main Loop	3 sequential specialized calls (`0x322`)	1 singular unified bulk call (`0x108`)
Pipeline Storage Overhead	0 bytes (No `vptr`/`vtable` allocation)	0 bytes (No `vptr`/`vtable` allocation)
Data Segment Register Setup	Loaded sequentially per invocation	Single base-pointer chunk load